
# Agentic Workflows Hackathon

Choose a problem worth solving and use agents to create something people would genuinely find useful.

## Welcome

Welcome to the **micro1 Agentic Workflows Hackathon**. Choose a problem worth solving and use agents to create something people would genuinely find useful. Keep it practical, share what you learn and have fun.

## Your challenge

Pick a **specific and meaningful problem you understand**. Use agents to solve it and show through clear evidence that your solution improves the way the task is handled today.

Start by explaining who has the problem. Describe the bottleneck they face and why solving it would be valuable in practice. The goal is to create something a real person would want to use.

KEEP FOUR QUESTIONS IN MIND

01

### Who has this problem?

02

### What bottleneck makes it worth solving?

03

### Does the agent solve it well?

04

### Can another person reproduce the result?

# How agents can help

Use whichever agent capabilities help solve the problem well. One solution may improve when the agent receives better context or better tools. Another may use memory to carry important information forward. Verification can catch errors before they reach the user, while specialized skills can deepen the agent's ability in a particular task. Some solutions may benefit from orchestration across several agents.

Choose the approach that fits your problem. Judges focus on whether each design choice improves the solution and helps the agent reach the goal reliably. Purposeful choices matter more than the number of components.

# Show how the solution improved

Create a **simple baseline** that represents a reasonable basic way to handle the task before using your solution. For example:

- One direct prompt with basic instructions.
- One general purpose agent with basic tools.
- A simple script or template.
- The manual process people use today.

Keep the comparison fair by giving the baseline and final solution the same task and evaluation cases. Explain any meaningful difference in the resources available to each one.

Use the final baseline comparison to show the size of the overall improvement. Use the changelog to explain where that improvement came from. Together, they tell the complete story of your solution.

# Tell the story with an improvement changelog

Create a short changelog that tells the story of how your solution evolved. Start with the simple baseline and follow the journey through to the final result. This makes it clear how each meaningful change contributed.

Add one entry for every important experiment. Explain what you tried and why you tried it. Then show the result using the same evaluation method whenever possible and share what you decided to do next. Include experiments you later removed and explain what they taught you about the problem.

THE PROGRESSION BELOW IS AN EXAMPLE. REPLACE IT WITH THE CHANGES YOUR PROJECT ACTUALLY MADE.

| STAGE | WHAT YOU TRIED AND WHY | EVIDENCE | DECISION / LEARNING |
|---|---|---|---|
| **Baseline** | Started with [basic approach] | `[baseline result]` | Established the starting point |
| **Iteration 1** | Added a skill to address [issue] | `[new result]` | [kept, revised or removed] |
| **Iteration 2** | Added verification after observing [failure] | `[new result]` | [kept, revised or removed] |
| **Iteration 3** | Changed orchestration to improve [goal] | `[new result]` | [kept, revised or removed] |
| **Final** | Combined the changes that worked | `[final result]` | Identified the main contribution |

# How to evaluate your solution

Choose one primary metric that reflects what success means to the user. For a developer, that might be how many tests pass. An operations team may care more about saving time or reducing cost, while a forecasting team may focus on calibration. Pick the measure that best captures the improvement your solution promises.

Before running the evaluation, define what a good final result looks like for the intended user. Use the same cases for the baseline and final solution, then share the complete results. Ten or more cases is a good target when the task allows it. Include one challenging case and explain what it revealed.

A SIMPLE FORMAT YOU CAN USE

| METRIC | SIMPLE BASELINE | AGENT SOLUTION | CHANGE |
|---|---|---|---|
| **Primary outcome** | [value] | [value] | [change] |
| **Human time per task** | [value] | [value] | [change] |
| **Cost per task** | [value] | [value] | [change] |

> You run this evaluation yourself. If the format above fits your task poorly, design your own clear scoring rubric and propose it, so the judges can use it to assess your workflow.

# How judging works

Projects receive a score out of 100 points. Each row describes what strong work looks like.
Use the question at the end to check your own project before submitting.

| CRITERION | POINTS | WHAT STRONG WORK LOOKS LIKE |
|---|---|---|
| **Problem & User Value** | 15 | A strong project solves a meaningful problem for a clearly defined user.<br>**Ask yourself:** Who experiences the bottleneck and why does solving it matter? |
| **Agent Solution & Engineering** | 30 | A strong solution uses agents purposefully and is technically sound. Better context or tools may improve one project, while memory, verification, skills or orchestration may improve another.<br>**Ask yourself:** Which design choices helped the agent solve the problem? |
| **End to End Quality** | 20 | A strong solution completes a realistic and self contained execution and produces a final result the user can use, with the finish of something a person would sign their name to rather than an obvious AI generated draft.<br>**Ask yourself:** Would the intended user consider this output high quality, or does it read as clearly AI generated? |
| **Measured Improvement** | 15 | A strong report demonstrates gains over a fair baseline and uses the changelog to connect each iteration with evidence.<br>**Ask yourself:** Which changes truly improved the outcome? |
| **Reproducibility** | 15 | A reproducible project gives another person a clear path to run the solution and baseline and reach the main result.<br>**Ask yourself:** Could they do it from a clean environment? |
| **Hot Take / Insights** | 5 | A strong insight turns an observed failure mode into a practical lesson for building more reliable agents.<br>**Ask yourself:** What did you learn and how would it change what you build next? |
| **Total** | 100 | |

# Ground rules

These rules are baseline requirements for every eligible project.

01 You are welcome to build with tools and components you already know.

02 Make it clear what existed before the competition and what you added.

03 Use every tool and component according to its license and service terms.

04 Keep consequential actions controlled through a sandbox or simulation. Add human approval before the action happens.

05 Make a qualified human reviewer part of any solution that could significantly affect someone.

06 Choose a legal and ethical use case that treats people and their data responsibly.

07 Use information you are allowed to share. Public or synthetic data are usually the easiest options. Approved anonymous data also works.

08 Keep credentials and private information outside the submission.

09 Connect every claim about your results to the evidence you submit.

10 Give judges enough access to run the project and reproduce the main result.

# Final deliverables

Submit your deliverable with these four items.

## 01 Complete solution code and improvement changelog

Share the full project and everything required to run it. Include the code as well as the instructions that shape each agent. Use the README to introduce the intended user and explain their current bottleneck. Then describe why solving it is valuable. Add a clearly labeled **Improvement Changelog** using the structure above. Give every meaningful iteration its own entry and connect it to the evidence that guided your next decision. Close with the main failure mode and your hot take.

## 02 Reproduction guide

Write for someone starting from a clean environment. Walk them through setup and provide the exact commands for the solution, baseline and evaluation. Explain which data is required and what output to expect. Share the relevant versions along with the approximate runtime and cost.

## 03 Solution video

Submit a video of up to **[5 minutes]**. Begin with the problem and simple baseline, then walk through one realistic execution from start to finish. Show the final comparison and briefly explain the changelog. Highlight the change that contributed most as well as one experiment you removed.

## 04 Agent trajectories

Include representative trajectories for **every agent you used**. Make each trajectory easy to follow from the agent instructions to the final result. Show what the agent did and how its tools responded. Capture the feedback that shaped its next step as well as any retries or human checkpoints.

APPENDIX

## Three examples for reference

# Code analysis: is this repository actually good?

### 01 Who has this problem?

One possible scenario could be a team considering the purchase of a private repository and they need to know what the code is worth. Since they did not build it, there must be a way to reliably sense its quality before agreeing on a fair price.

### 02 What bottleneck makes it worth solving?

If you think in a code repository, a README file or working demo reveals little about the quality of the actual code. The buyer must understand an unfamiliar codebase, also run the build and tests, inspect the architecture and dependencies and assess technical debt and maintenance risks. There is also relevant evidence in pull requests or open issues and reviewers may interpret the same signals differently. If you don't have a repeatable method, the valuation can depend on an incomplete or inconsistent judgment.

### 03 Does the agent solve it well?

A useful system could analyze the repository and give the buyer a clear quality assessment before they negotiate the price. The team still has to define what "good" means and how code quality should influence the valuation. One way to test it is to have qualified reviewers rank ten approved codebases with a shared rubric, then give the same codebases and rubric to the agent and to a simple baseline. Does the agent come closer to the reviewers' order, and can it explain each position with evidence?

### 04 Can another person reproduce the result?

Use approved repositories and document the exact setup, commands, tool versions and expected output for both the baseline and the agent. Tie every score to a file, test result or build output. A second person starting from a clean environment should be able to run the workflow on the same codebases and reproduce the assessment and relative ranking.

# Candidate evaluation: should we hire this person?

### 01 Who has this problem?

Think on recruiters and hiring managers that need to decide whether a candidate is right for a role. The evidence they need is spread across the job description, the target profile, the candidate's CV, interview records and any completed assessments.

### 02 What bottleneck makes it worth solving?

If you are actually reviewing each source in isolation, it is pretty easy to miss contradictions or give one signal too much weight. A candidate may look perfect at the beginning even when the evidence does not fully line up. Even more, if you are actually suspecting the candidate cheated then it makes the decision more sensitive because a warning sign alone is not proof of it.

### 03 Does the agent solve it well?

Potentially, an agent could bring the evidence into one review, connect job requirements to demonstrated skills, check stated experience against approved sources and explain any discrepancies. The actual recommendation should make its evidence and uncertainty visible while leaving the final decision to a qualified reviewer.

### 04 Can another person reproduce the result?

You can use approved or synthetic candidate cases so the evaluation does not depend on private information. Run the baseline and the agent on the same cases including one candidate with conflicting signals. Report every result, including failures, and trace each score or concern back to its source. A second reviewer should be able to reproduce the assessment from the same material without big discrepancies or changes on the resolution.

# Podcast translation: can every version still feel like the same show?

### 01 Who has this problem?

Think on podcast creators and teams that are responsible for how a show sounds in every language. Basically, each translated episode must remain consistent with the episodes that came before it.

### 02 What bottleneck makes it worth solving?

The big problem is context can span hours of audio, multiple speakers, earlier episodes and prior translation choices. One episode may sound fine in isolation while inconsistencies accumulate across the series. Some challenges this could have is something like a speaker's name may be pronounced differently, a recurring phrase may be translated differently from one episode to the next or a joke may lose its meaning because an earlier reference was handled another way. Each sentence can be correct while the series as a whole no longer feels coherent.

### 03 Does the agent solve it well?

A strong solution would translate across episodes and languages while keeping speaker identity, pronunciation, recurring terms, tone and prior decisions consistent over all of it. Whether it produces transcripts, subtitles or dubbed audio the result should preserve the meaning and timing of the original while sounding natural in the target language.

### 04 Can another person reproduce the result?

It is important to define the evaluation before running it. Choose a fixed set of episodes and target languages then use the same inputs for the baseline and the agent. Include one case that depends on a recurring detail. Each translation choice should point back to the source audio or approved material, such as show notes or a glossary. Anyone should be able to rerun the evaluation and check the result.